

Sistemi Informativi su Web aa 2025/2026

Appello Settembre 2026

Progetto assegnato dal Docente

1. Obiettivo

Il progetto assegnato dal docente consiste nella realizzazione di un piccolo sistema informativo su Web per la gestione di festival cinematografici.

Il sistema deve essere implementato utilizzando:

- Spring Boot (backend)
- JPA / Hibernate (persistenza)
- PostgreSQL (o altro RDBMS)
- Thymeleaf (frontend)
- React (almeno per una parte del frontend)

L'obiettivo è progettare e implementare un sistema informativo completo, con particolare attenzione a:

- architettura a livelli
- gestione delle transazioni
- accesso efficiente ai dati
- separazione delle responsabilità

2. Scenario

Il sistema gestisce uno o più festival cinematografici. Per ogni festival vengono registrati i film partecipanti, i registi, le sale cinematografiche e le proiezioni programmate.

I film possono partecipare a uno o più festival e, nell'ambito di ciascun festival, possono essere presentati attraverso una o più proiezioni.

Il sistema distingue tra utenti generici, utenti registrati e utenti amministratori. Gli utenti registrati possono inserire recensioni relative ai film, mentre gli amministratori possono accedere alle funzionalità di gestione del festival.

3. Entità principali

Festival

Attributi minimi:

- nome
- anno
- città
- dataInizio
- dataFine

- descrizione

Relazioni minime:

- un festival presenta più film
- un festival prevede più proiezioni

Film

Attributi minimi:

- titolo
- anno
- durata
- genere
- paeseProduzione

Relazioni minime:

- un film può partecipare a uno o più festival
- ogni film ha un regista
- un film può avere più proiezioni
- un film può avere più recensioni

Regista

Attributi minimi:

- nome
- cognome
- dataNascita
- nazionalità

Relazioni minime:

- un regista può aver diretto più film

Sala

Attributi minimi:

- nome
- indirizzo
- capienza

Relazioni minime:

- una sala può ospitare più proiezioni

Proiezione

Attributi minimi:

- data
- ora
- stato (ad esempio SCHEDULED, COMPLETED, CANCELLED)

Relazioni minime:

- una proiezione appartiene a un festival
- una proiezione riguarda un film
- una proiezione si svolge in una sala

Recensione

Attributi minimi:

- testo
- voto
- data

Relazioni minime:

- una recensione riguarda un film
- una recensione è scritta da un utente registrato

Un utente può inserire al massimo una recensione per uno stesso film.

Utente

Attributi minimi:

- username
- password
- ruolo

4. Casi d'uso richiesti

4.1 Funzionalità pubbliche

Il sistema deve consentire almeno:

- visualizzazione dell'elenco dei festival
- visualizzazione del dettaglio di un festival
- visualizzazione dei film partecipanti a un festival
- visualizzazione del programma delle proiezioni
- visualizzazione del dettaglio di un film
- visualizzazione dei dati del regista di un film
- visualizzazione delle recensioni di un film

Il dettaglio di un festival deve permettere di accedere ai film e alle proiezioni associate.

Il dettaglio di un film deve mostrare almeno le informazioni sul film, il regista, i festival ai quali partecipa, le eventuali proiezioni e le recensioni.

4.2 Funzionalità utenti registrati

- inserimento di una recensione relativa a un film
- modifica di una propria recensione
- eliminazione di una propria recensione

Un utente può modificare o eliminare esclusivamente le recensioni di cui è autore. Non deve essere possibile inserire più di una recensione dello stesso utente per lo stesso film.

4.3 Funzionalità riservate all'amministratore

- creazione e modifica di un festival
- inserimento e modifica di un film
- inserimento e modifica di un regista
- associazione di un film a un festival
- inserimento e modifica di una sala
- programmazione di una proiezione
- modifica o cancellazione di una proiezione
- eliminazione di un film da un festival

Il sistema deve verificare la consistenza delle operazioni effettuate. Ad esempio, non deve essere possibile programmare due proiezioni nella stessa sala nello stesso intervallo temporale.

5. Requisiti di sicurezza

L'applicazione deve prevedere autenticazione e autorizzazione.

Ruoli

- USER: accesso alle funzionalità riservate agli utenti registrati
- ADMIN: accesso alle funzionalità di amministrazione e gestione del sistema

Requisiti minimi

- login con username e password
- protezione delle funzionalità amministrative
- protezione delle operazioni relative alle recensioni
- verifica che un utente possa modificare o cancellare esclusivamente le proprie recensioni

6. Architettura dell'applicazione

L'applicazione deve essere strutturata secondo una architettura a livelli.

Persistence Layer

- entità JPA
- repository JPA per l'accesso ai dati

Service Layer

- implementa i casi d'uso
- coordina uno o più repository
- contiene la logica di business
- gestisce le transazioni

La logica applicativa non deve essere implementata nei Controller.

Controller Layer

- gestisce le richieste HTTP
- valida gli input quando necessario
- delega l'esecuzione dei casi d'uso al Service Layer

- prepara i dati destinati al frontend

7. Gestione delle transazioni

I metodi del Service Layer devono essere opportunamente annotati con `@Transactional`.

Occorre distinguere tra:

- operazioni di sola lettura
- operazioni di aggiornamento

La scelta del livello di isolamento deve essere coerente con il caso d'uso.

Gli studenti devono individuare almeno un'operazione che coinvolga più entità e repository e discuterne il comportamento transazionale.

Ad esempio, la programmazione di una nuova proiezione può richiedere:

1. recupero del festival
2. recupero del film
3. recupero della sala
4. verifica della disponibilità della sala
5. creazione della proiezione
6. aggiornamento delle associazioni coinvolte

L'operazione deve essere atomica: in caso di errore, lo stato del sistema deve rimanere consistente.

8. Prestazioni e accesso ai dati

Le operazioni di accesso ai dati devono essere progettate tenendo conto anche delle prestazioni.

8.1 Progettazione

Gli studenti devono prestare particolare attenzione a:

- strategie di fetch
- relazioni LAZY ed EAGER
- numero di query generate
- problema delle N+1 query
- caricamento di collezioni associate

Devono essere motivate le strategie adottate almeno per le associazioni principali del modello.

8.2 Analisi sperimentale dell'accesso ai dati

Per almeno un caso d'uso significativo, gli studenti devono predisporre un semplice script o test eseguibile che permetta di osservare il comportamento di JPA/Hibernate nell'accesso ai dati.

Lo script deve poter essere eseguito durante la prova orale e deve confrontare almeno due diverse strategie di accesso ai dati, ad esempio:

- accesso tramite associazioni LAZY
- query con join fetch
- utilizzo di EntityGraph
- eventuale utilizzo di associazioni EAGER, quando opportuno

Possibili casi d'uso sono, ad esempio:

- caricamento di tutti i film di un festival con i relativi registi
- caricamento del programma di un festival con film e sale
- caricamento di un insieme di film con le relative proiezioni

Requisiti dello script

Lo script deve:

1. eseguire lo stesso caso d'uso utilizzando almeno due strategie differenti
2. operare sullo stesso insieme di dati
3. mostrare per ciascuna strategia almeno il numero di query SQL eseguite, il tempo complessivo di esecuzione e il numero di oggetti o record caricati
4. produrre un output semplice e leggibile sulla console

Un possibile output è:

```
=== Test accesso ai film del festival ===
```

```
Strategia 1: LAZY
```

```
Film caricati: 100
```

```
Query SQL: 101
```

```
Tempo: 84 ms
```

```
Strategia 2: JOIN FETCH
```

```
Film caricati: 100
```

```
Query SQL: 1
```

```
Tempo: 21 ms
```

Non è richiesta una infrastruttura di benchmarking sofisticata. È sufficiente un semplice test Java, un test JUnit o un componente eseguibile all'interno dell'applicazione Spring Boot.

Lo script deve essere poter essere eseguito senza modifiche sostanziali al codice.

Discussione durante l'esame

Durante la prova orale lo studente potrà essere chiamato a:

- eseguire lo script
- spiegare le query generate da Hibernate
- individuare un eventuale problema di N+1 query
- spiegare le differenze tra le strategie confrontate
- modificare la strategia di accesso ai dati
- prevedere come cambierebbe il comportamento al crescere del numero di entità

L'obiettivo non è effettuare un benchmark rigoroso delle prestazioni, ma dimostrare la capacità di osservare, comprendere e discutere il comportamento di JPA/Hibernate.

9. Frontend

L'applicazione deve includere almeno una parte realizzata utilizzando React. Il resto dell'interfaccia può essere realizzato utilizzando Thymeleaf.

La parte React deve ottenere i dati dal backend attraverso una o più API REST realizzate con Spring Boot.

A titolo di esempio, può essere realizzata in React una delle seguenti funzionalità:

- programma delle proiezioni
- elenco e ricerca dei film
- dettaglio di un festival
- visualizzazione e inserimento delle recensioni

La scelta deve essere adeguatamente integrata nell'applicazione.

10. API REST

Il progetto deve includere almeno un insieme di endpoint REST utilizzati dal frontend React.

Gli endpoint devono essere progettati rispettando le convenzioni HTTP. Ad esempio:

- GET /api/festivals
- GET /api/festivals/{id}
- GET /api/festivals/{id}/movies
- GET /api/festivals/{id}/screenings
- GET /api/movies/{id}
- GET /api/movies/{id}/reviews

Quando appropriato, devono essere utilizzati GET, POST, PUT o PATCH e DELETE. Gli endpoint devono restituire opportuni codici di stato HTTP e gestire correttamente le condizioni di errore.

Non è richiesto che l'intera applicazione utilizzi un'architettura REST.

11. Requisiti minimi

- almeno 6 entità significative
- relazioni uno-a-molti, molti-a-uno e almeno una relazione molti-a-molti
- almeno 2 casi d'uso per ogni categoria di funzionalità indicata nella Sezione 4
- almeno un caso d'uso che coinvolga più entità
- autenticazione
- autorizzazione
- utilizzo di Spring Security
- gestione delle transazioni
- almeno una parte del frontend sviluppata in React
- almeno una API REST utilizzata dal frontend React
- validazione degli input
- gestione degli errori

12. Criteri di valutazione

- correttezza funzionale
- qualità del modello dati
- corretto mapping delle associazioni JPA
- uso corretto di JPA / Hibernate
- qualità del Service Layer
- corretta separazione delle responsabilità

- gestione delle transazioni
- gestione della sicurezza
- qualità delle API REST
- integrazione tra React e Spring Boot
- efficienza dell'accesso ai dati
- comprensione delle strategie di fetch
- qualità del codice
- chiarezza dell'interfaccia utente
- validazione dei dati
- gestione degli errori

13. Bonus facoltativi

- paginazione
- ricerca e filtri
- ricerca dei film per titolo, genere o regista
- ricerca delle proiezioni per data
- upload di locandine o immagini
- statistiche sulle recensioni
- documentazione delle API REST
- autenticazione con OAuth
- deployment su una piattaforma cloud

14. Prova orale

Durante la prova orale lo studente dovrà dimostrare di conoscere il progetto realizzato e di comprenderne le principali scelte architetturali e implementative.

Il docente chiederà allo studente di modificare o estendere il progetto durante l'esame.

Le modifiche potranno riguardare, ad esempio:

- aggiunta di un attributo o di una nuova relazione tra entità
- modifica di un mapping JPA
- implementazione di una nuova query
- aggiunta di un caso d'uso
- modifica di una regola di business
- aggiunta o modifica di un metodo del Service Layer
- modifica della gestione transazionale
- aggiunta di un endpoint REST
- modifica della configurazione di Spring Security
- modifica della strategia di fetch
- individuazione e correzione di un problema N+1
- modifica di una componente React

Lo studente dovrà essere in grado di motivare le modifiche effettuate e di discuterne le conseguenze sul modello dei dati, sull'architettura e sul comportamento dell'applicazione.

15. Consegna

Inviare un messaggio di posta elettronica all'indirizzo **siw.roma3@gmail.com** entro le **ore 12:00 del 13 settembre 2026**.

ATTENZIONE: non inviare il messaggio all'indirizzo di posta del docente.

L'oggetto del messaggio deve iniziare con la stringa [Settembre 2026 PROGETTO DOCENTE] seguita da cognome e matricola dello studente. Esempio: per lo studente Francesco De Rossi con matricola 123456, l'oggetto del messaggio deve essere:

[Settembre 2026 PROGETTO DOCENTE] De Rossi 123456.

Nel corpo del messaggio riportare:

- indirizzo GitHub (o altra piattaforma Git) del progetto con il codice sorgente
- eventuali malfunzionamenti noti, ma non risolti
- considerazioni generali sull'esperienza

Nota: se lo studente non supera l'esame o rifiuta il voto, all'esame successivo dovrà portare un nuovo progetto assegnato dal docente circa 20 giorni prima della prova orale.

Progetto personale

1. Obiettivi

Progettare il proprio sistema, definendone casi d'uso, modello di dominio (con indicazioni utili alla progettazione dello strato di persistenza).

Implementare almeno 6 casi d'uso:

- almeno uno che richieda operazioni di inserimento dei dati di una entità
- almeno uno che richieda aggiornamento (update) dei dati di una entità
- almeno uno che richieda cancellazione di una (o più) entità
- almeno due che richiedano operazioni di lettura dei dati di una o più entità

NB: autenticazione e registrazione dell'utente non sono considerati casi d'uso.

Il sistema deve essere implementato utilizzando:

- Spring Boot (backend)
- JPA / Hibernate (persistenza)
- PostgreSQL (o altro RDBMS)
- Thymeleaf (frontend)
- React (almeno per una parte del frontend)

Il deploy su una piattaforma cloud (ad esempio, AWS, Azure, o Heroku) e l'autenticazione tramite OAuth saranno considerati un plus.

2. Consegna

Inviare un messaggio di posta elettronica all'indirizzo **siw.roma3@gmail.com** entro le ore 12:00 del giorno 13 settembre 2026.

ATTENZIONE: non inviare il messaggio all'indirizzo di posta del docente.

L'oggetto del messaggio deve iniziare con la stringa [Giugno/Luglio 2026 PROGETTO PERSONALE] seguita da cognome e matricola dello studente. Esempio: lo studente Francesco De Rossi con matricola 123456 che si prenota per l'appello di giugno l'oggetto del messaggio deve essere:

[Settembre 2026 PROGETTO PERSONALE] De Rossi 123456.

Nel corpo del messaggio riportare:

- indirizzo github (o altra piattaforma git) del progetto (con codice sorgente);
- eventuali malfunzionamenti noti, ma non risolti.

Nota: se lo studente non supera l'esame o rifiuta il voto, all'appello successivo potrà portare lo stesso progetto personale.